

Technical Assessment

Time expectation: Take up to 48 hours to submit — work around your schedule.

Tech stack: JavaScript/TypeScript throughout — **React** for the frontend, **Node.js** for the backend. Please don't use Python or another backend language. Third-party STT/TTS/LLM APIs are fine regardless of what language their SDK is written in since you're just calling them over HTTP/SDK from your Node backend.

AI tools: Fully allowed. Use whatever you'd normally use on the job — Copilot, ChatGPT, Claude, Cursor, whatever.

What You're Building

A web app where a user can have a live voice conversation with an AI agent that conducts a basic health screening call — similar in spirit to conversational voice-AI demos like Sarvam AI's. The user talks, the AI asks basic health questions like a real intake call would, and once the call ends, the app shows a structured health report summarizing what was discussed.

Must-Have Functionality

1. The Call

- A "Start Call" button that begins a live voice conversation between the user and an AI agent, and an "End Call" button that ends it.
- The AI should greet the user, then ask a handful of basic health-screening questions one at a time — e.g. name, main concern/symptom, how long it's been going on, severity, any other related symptoms. This should feel like a real, adaptive conversation, not a fixed script read top to bottom regardless of what the user says (a reasonable follow-up question when an answer is vague is a good example of this).
- The conversation should work in **Hindi or English** at minimum (your choice which one you build for). **Bonus:** auto-detect which language the user is speaking and respond in kind, or handle a mid-call language switch.
- The call transport should be real-time-oriented — WebRTC, WebSockets, or similar. Not "record the whole conversation and upload one audio file at the end."

2. The Pipeline You'll need some combination of: speech-to-text, an LLM driving the conversation, and text-to-speech. Use any APIs you have access to (Sarovam AI, OpenAI/Whisper, Deepgram, AssemblyAI, ElevenLabs, Google Cloud Speech, Anthropic, or anything else) — note in your README what you used.

A fully real-time, full-duplex, no-latency call is genuinely hard to get right in a take-home window, and we're not expecting that. A "user speaks their turn → sent for processing → AI responds with audio → back to the user" push-to-talk-style flow is a completely acceptable and expected way to build this. What matters is that turn-taking and conversation state are handled correctly — the AI should remember what's already been asked and answered, not repeat questions or lose the thread.

3. The Report

- Once the call ends, generate and display a **structured health report** — a clear summary of the call: the main concern, key symptoms mentioned, duration, severity, anything the AI flagged as worth following up on. This should read like something a doctor could glance at, not a raw transcript dump.
- Handle a short or incomplete call gracefully — if the user ends the call after one exchange, the report shouldn't crash or show garbage; it should reflect that limited information was collected.

Nice-to-Have (pick any, entirely optional)

- Auto language detection/switching (if you didn't already build this into Must-Have)
- Handling "barge-in" — the user starts speaking while the AI is still talking
- Graceful handling of silence, background noise, or the STT returning nothing useful

What We're Evaluating

- **Does the call actually work.** This is the main thing — we want to experience a real conversation, not just read code that claims to do one.
- **Pipeline architecture.** How you structured the STT → LLM → TTS flow, and how cleanly the real-time transport is handled.
- **Conversation state management.** Does the AI track what's already been asked/answered across turns, or does it lose context.
- **Failure handling.** What happens on silence, unclear audio, or an API call failing mid-conversation — does the call recover or just die.
- **Report quality.** Does it meaningfully synthesize a messy spoken conversation into something structured and useful, rather than just echoing the transcript back.

We do not expect production polish, perfect latency, or flawless language handling. We expect a working, honestly-scoped slice of a real conversational AI feature, plus clarity on what you'd improve with more time.

Submission

You have **48 hours from the time you receive this email** to complete and submit the assessment.

Please **reply to this email within 48 hours** with the link to your **GitHub repository**.

- Make sure the GitHub repository is set to **Public** so that we can access and review your submission.
- Include all necessary setup instructions, including **API keys, services, environment variables, or other requirements** needed to run the project, in the repository's README.
- **Any submission received after the 48-hour deadline will be automatically rejected.**